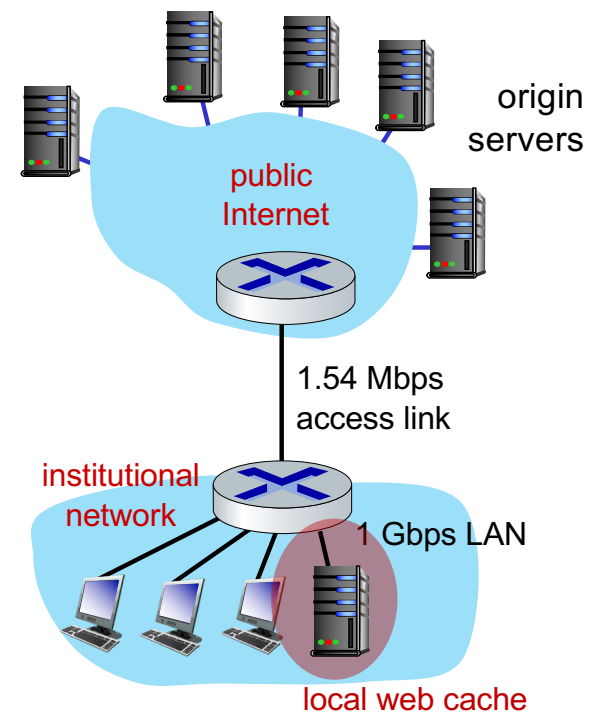


Quiz: Web Cache and Queuing

Scenario:

- access link rate: 1.54 Mbps
- public Internet delay: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Q: What's the required cache hit rate for access link utilization to be below 0.5?



Quiz: Web Cache and Queuing

Scenario:

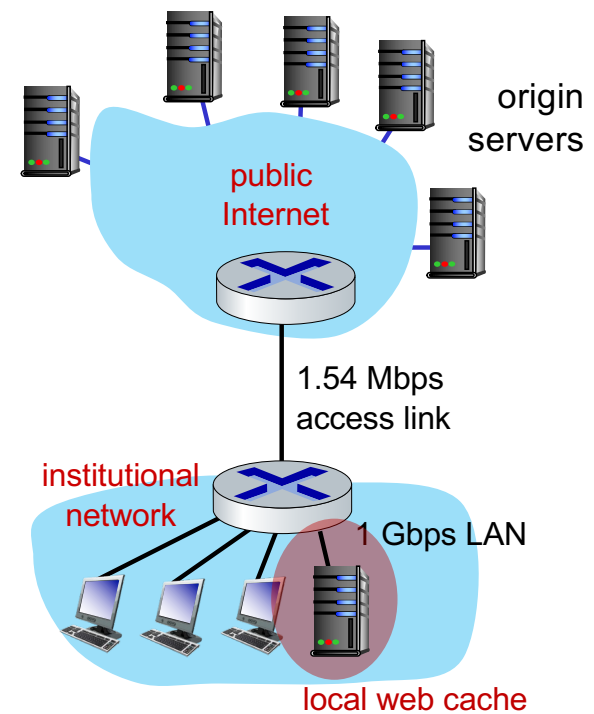
- access link rate: 1.54 Mbps
- public Internet delay: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers:
 - 15/sec
 - avg data rate to browsers: 1.50 Mbps

Q: What's the required cache hit rate for access link utilization to be below 0.5?

A: hit rate is α , then we have:

$$\frac{(1 - \alpha) * 1.5}{1.54} \leq 0.5$$

so $\alpha \geq 0.5$



Video Calls

- Is DASH a good option? **NO**
- Latency requirement: sub-second
 - DASH: 1-2 second for segment-based delivery, seconds of end delay due to buffering
- Continuous push-based content delivery
 - DASH: client-pull
- Two-way communication
 - DASH: one-to-many broadcast
- Interactive congestion control

Video Calls Use WebRTC

- WebRTC: a collection/framework of protocols, APIs, and standards for real-time communication
- Browser APIs
- Transport & security protocols
- Codecs formats
- Real-time congestion control
- Also used in gaming, interactive live streaming, remote desktop, AR streaming, and more

Assignment 1 Reminders

- **Gradescope** for submission
- Do **NOT** include files of downloaded video chunks in your submission
- TESTING document: think of representative test cases (bandwidth files, alpha values)
- README: document assumptions, how to run your code, corner cases or anything incomplete
- **Start early**, visit TA office hours, and enjoy the assignment!

Application layer: overview

- Principles of network applications
- Web and HTTP
- video streaming and content distribution networks
- socket programming with UDP and TCP
- The Domain Name System DNS
- P2P applications

What is DNS for?

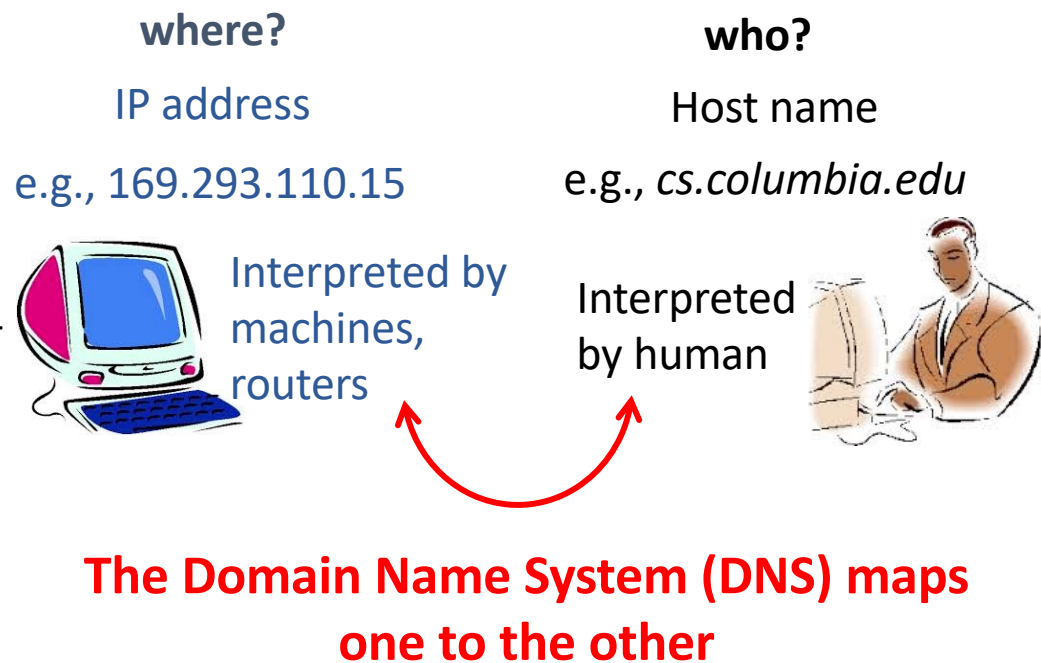
people: many identifiers:

- SSN, name, passport #

Internet hosts, routers:

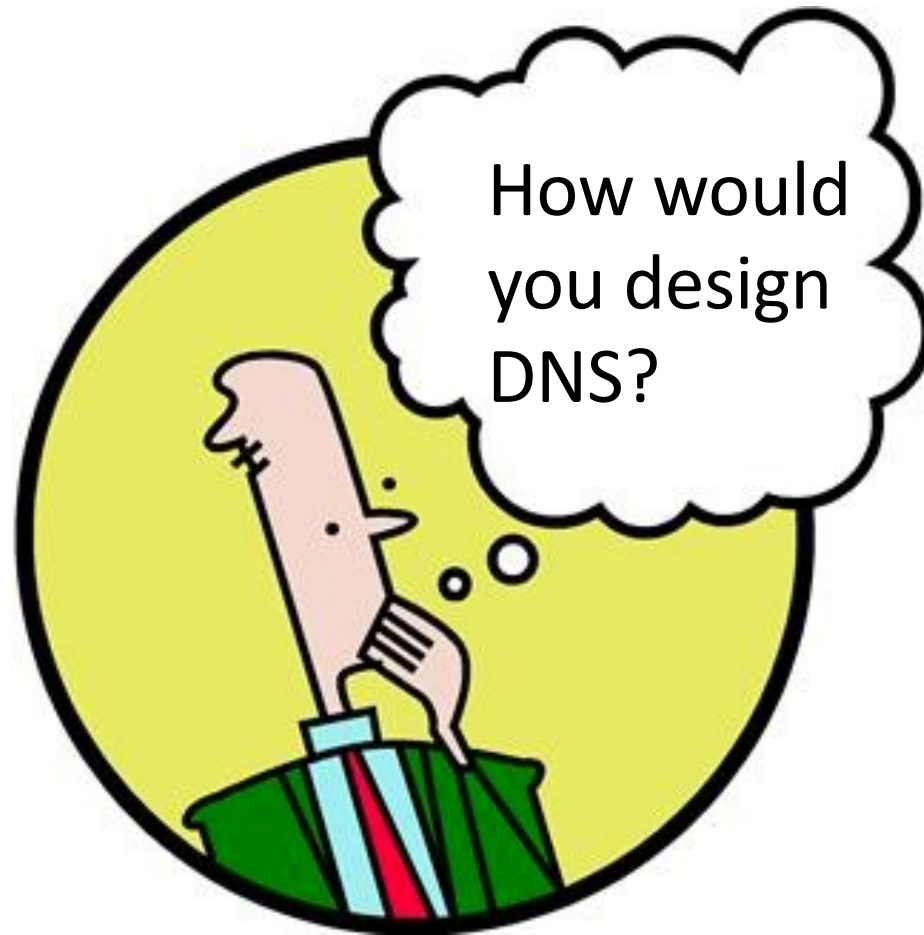
- IP address (32 bit) - used for addressing datagrams
- “name”, e.g., cs.columbia.edu - used by humans

Q: how to map between IP address and name, and vice versa ?



Why Separating Naming and Addressing?

- Can hide address change from users
 - e.g., move `www.columbia.edu` to a new IP address
- One name can map to multiple IP addresses
 - Multiple replicas of CNN servers
- Map a name to a different IP address based on location
 - e.g., return the IP of the nearest web server to reduce latency or return different content
- Multiple names for an address
 - e.g., aliases like `web.cs.dartmouth.edu` and `cs.dartmouth.edu`



How would
you design
DNS?

Solution #1: Local File

- Save all the host-address mappings in a local file
- That's actually what people did in the beginning!
 - Saved as a hosts.txt file under /etc/hosts
 - Maintained by Stanford Research Institute (SRI)
 - Email changes to SRI, download the file regularly
- **Problem: does not scale**
 - SRI cannot handle the load as the Internet grew



Solution #2: Central Server

- All mappings are stored at a central server
- Query the central server
- Problems?
 - Single point of failure
 - High traffic volume
 - Comcast DNS servers alone: 600B DNS queries/day
 - Akamai DNS servers alone: 2.2T DNS queries/day
 - Distant central database → query time can be long
 - Single point of update
- **Need distributed solutions!**



DNS: structure, services

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol*: hosts, DNS servers communicate to *resolve* names (address/name translation)
 - core Internet function, **implemented as application-layer protocol**
 - complexity at network's "edge"

DNS services:

- hostname-to-IP-address translation
- host aliasing
 - canonical, alias names
- mail server aliasing
- load distribution

Thinking about the DNS

humongous distributed database:

- ~ billion records, each simple

handles many *trillions* of queries/day:

- *many* more reads than writes
- *performance matters*: almost every Internet transaction interacts with DNS - msec count!

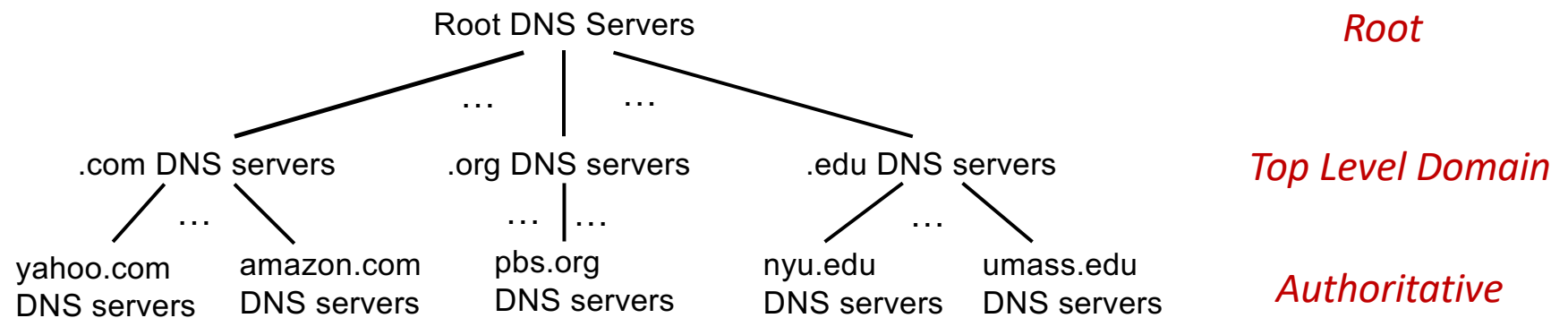
organizationally, physically decentralized:

- millions of different organizations responsible for their records

“bulletproof”: reliability, security



DNS: a distributed, hierarchical database

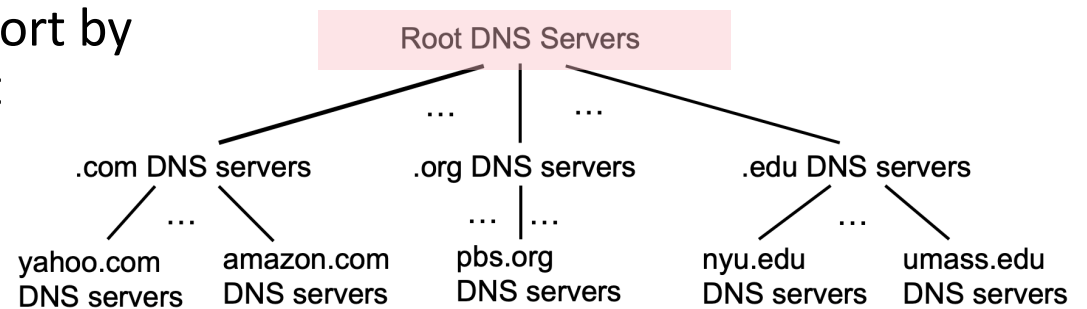


Client wants IP address for `www.amazon.com`; 1st approximation:

- client queries root server to find .com DNS server
- client queries .com DNS server to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for www.amazon.com

DNS: root name servers

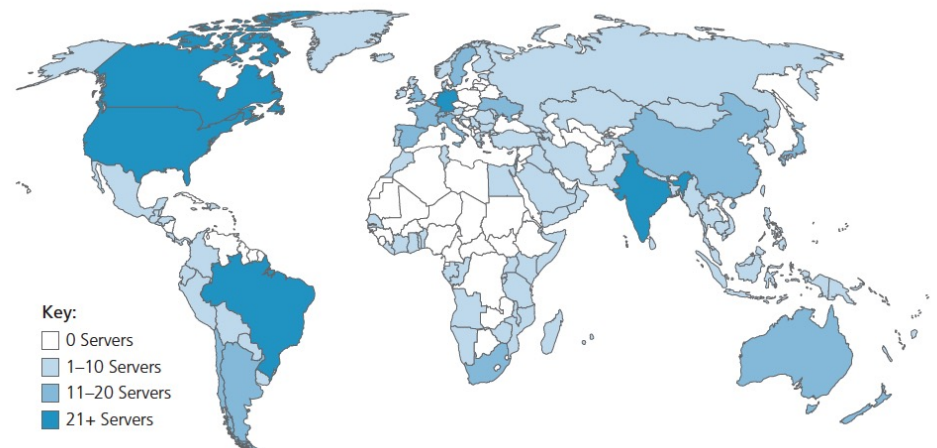
- official, contact-of-last-resort by name servers that can not resolve name



DNS: root name servers

- official, contact-of-last-resort by name servers that can not resolve name
- *incredibly important* Internet function
 - Internet couldn't function without it!
 - DNSSEC – provides security (authentication, message integrity)
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

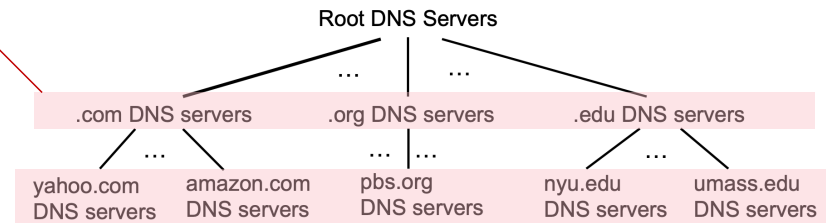
13 **logical** root name “servers”
worldwide each “server” replicated
many times (~200 servers in US)



Top-Level Domain, and authoritative servers

Top-Level Domain (TLD) servers:

- responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp
- Network Solutions: authoritative registry for .com, .net TLD
- Educause: .edu TLD



authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

Local DNS name servers

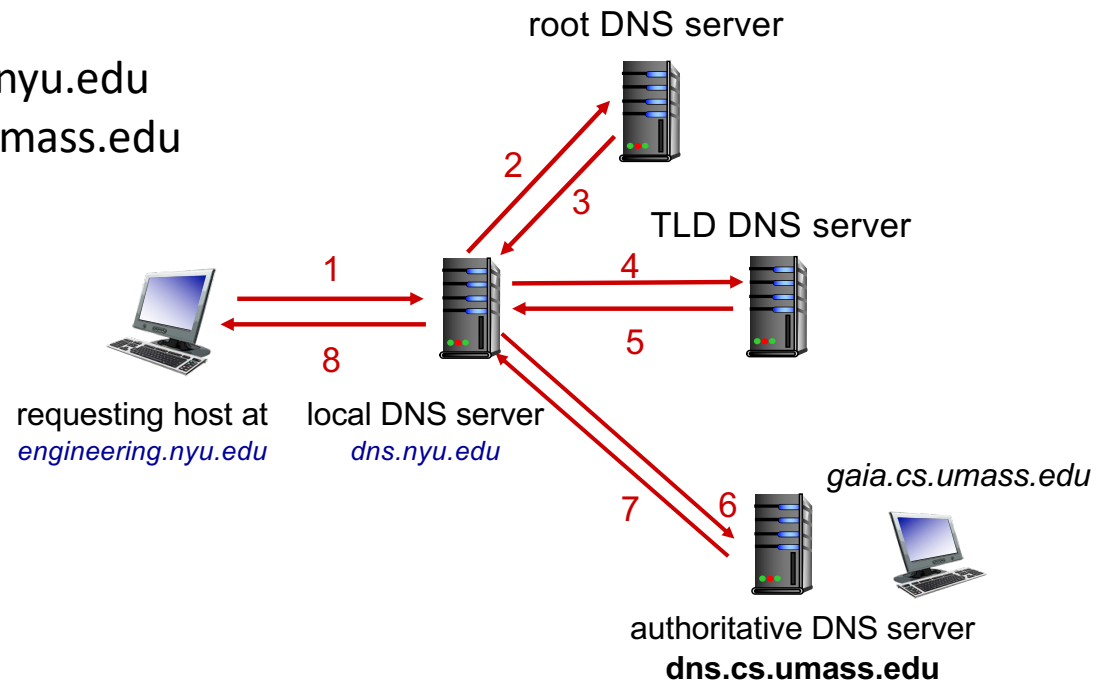
- A host sends DNS queries to its *local* DNS server
 - Local DNS server returns reply, answering:
 - from its local cache of recent name-to-address translation pairs (possibly out of date!)
 - forwarding request into DNS hierarchy for resolution
 - each ISP has local DNS name server; to find yours:
 - MacOS: `% scutil --dns`
 - Windows: `>ipconfig /all`
- local DNS server doesn't strictly belong to hierarchy

DNS name resolution: iterated query

Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Iterated query:

- contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”

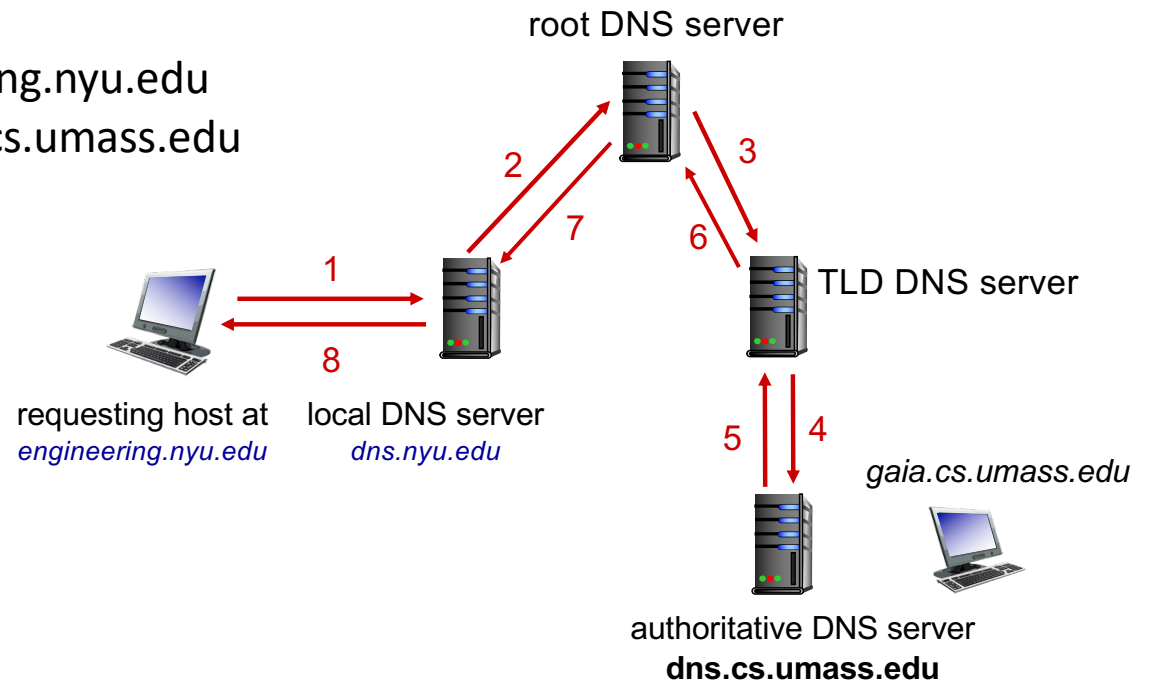


DNS name resolution: recursive query

Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



Caching DNS Information

- once (any) name server learns mapping, it *caches* mapping, and *immediately* returns a cached mapping in response to a query
 - caching improves response time
 - cache entries timeout (disappear) after some time (TTL)
 - TLD servers typically cached in local name servers
- cached entries may be *out-of-date*
 - if named host changes IP address, may not be known Internet-wide until all TTLs expire!
 - *best-effort name-to-address translation!*

DNS records

DNS: distributed database storing resource records (**RR**)

RR format: (name, value, type, ttl)

type=A

- name is hostname
- value is IP address

type=NS

- name is domain (e.g., foo.com)
- value is hostname of authoritative name server for this domain

type=CNAME

- name is alias name for some “canonical” (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- value is canonical name

type=MX

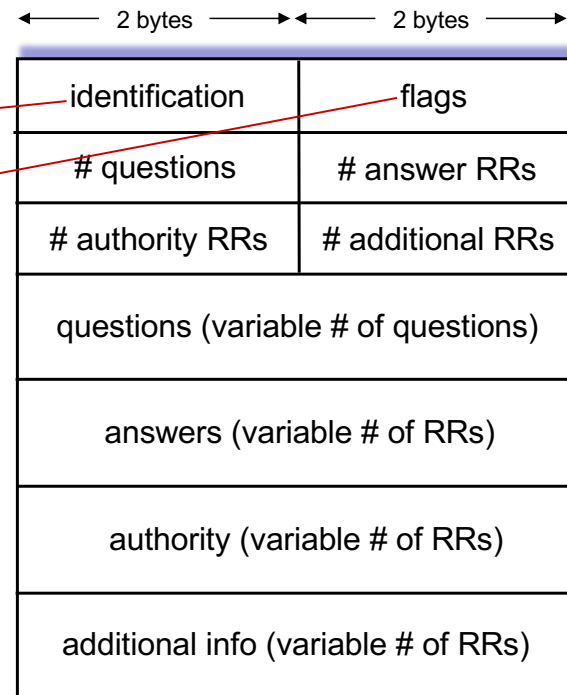
- value is name of SMTP mail server associated with name

DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:

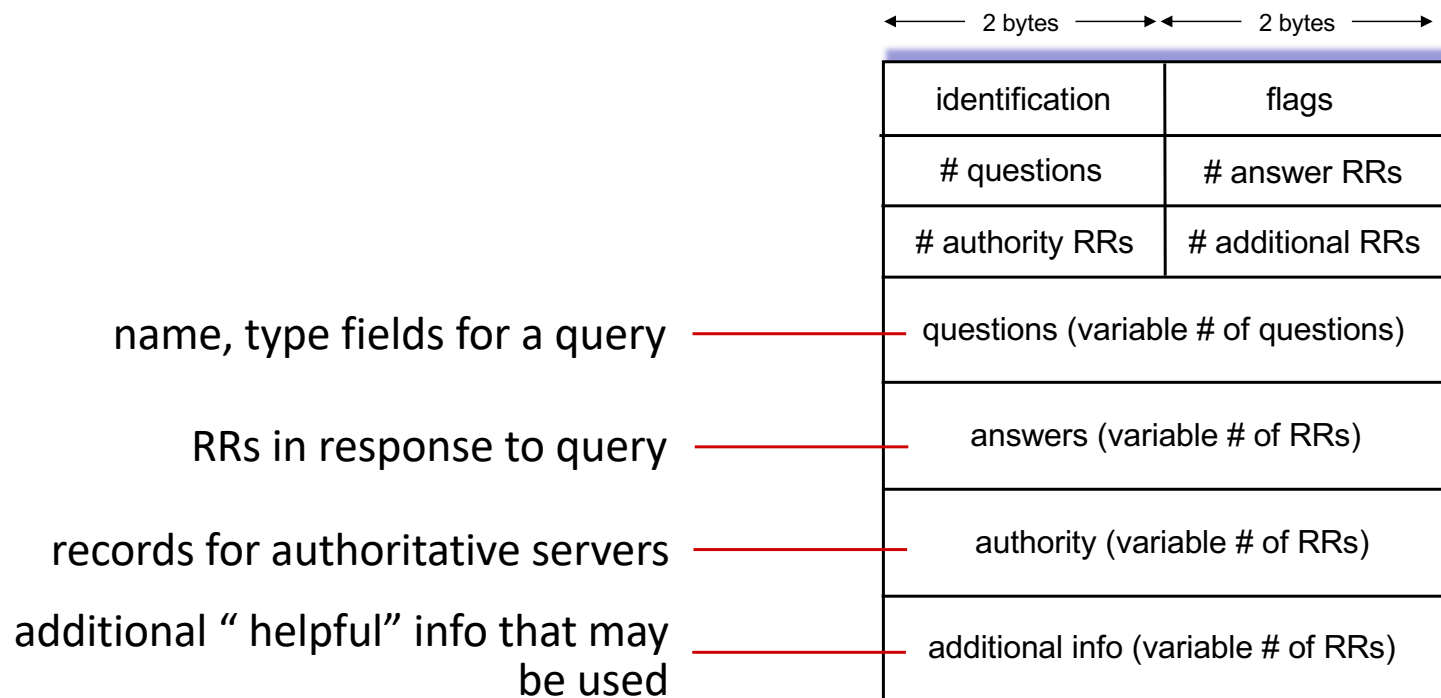
message header:

- **identification**: 16 bit # for query, reply to query uses same #
- **flags**:
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative



DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:



Getting your info into the DNS

example: new startup “Network Utopia”

- register name networkutopia.com at *DNS registrar* (e.g., Network Solutions)
 - provide names, IP addresses of authoritative name server (primary and secondary)
 - registrar inserts NS, A RRs into .com TLD server:
`(networkutopia.com, dns1.networkutopia.com, NS)`
`(dns1.networkutopia.com, 212.212.212.1, A)`
- create authoritative server locally with IP address 212.212.212.1
 - type A record for www.networkutopia.com
 - type MX record for networkutopia.com

DNS security

DDoS attacks

- bombard root servers with traffic
 - not successful to date
 - traffic filtering
 - local DNS servers cache IPs of TLD servers, allowing root server bypass
- bombard TLD servers
 - potentially more dangerous

Spoofing attacks

- intercept DNS queries, returning bogus replies
 - DNS cache poisoning
 - RFC 4033: DNSSEC authentication services
 - Each DNS zone has a pair of private/public key
 - DNS data signed by the zone owner

Utility of Querying DNS Servers

- Linux: **dig** command
 - dig grasshopper.cs.columbia.edu A (getting the IP address of a hostname)
 - dig cs.columbia.edu MX (getting MX record to find out mail server)
- Windows: **nslookup** command
- A webtool from Google:
<https://toolbox.googleapps.com/apps/dig/>

Quiz

Assume you are connecting to a web server using HTTP 1.1 w/ persistent connection.

Calculate the # of RRTs needed to fetch and receive an index.html with 5 embedded jpeg images. Assume each image can fit in one packet.